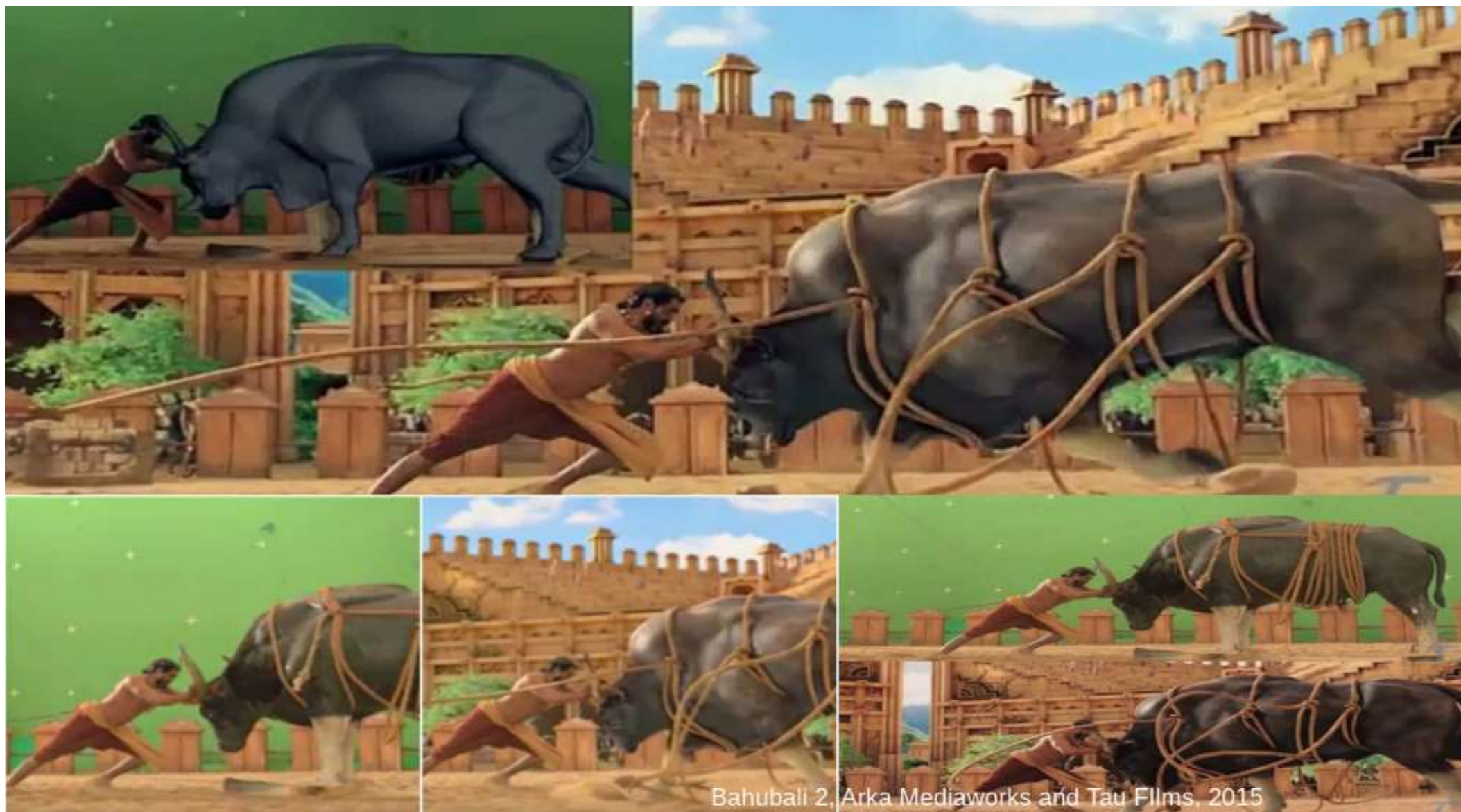




What is Computer Graphics?



Ernest and Celestine. Les Armateurs. 2013



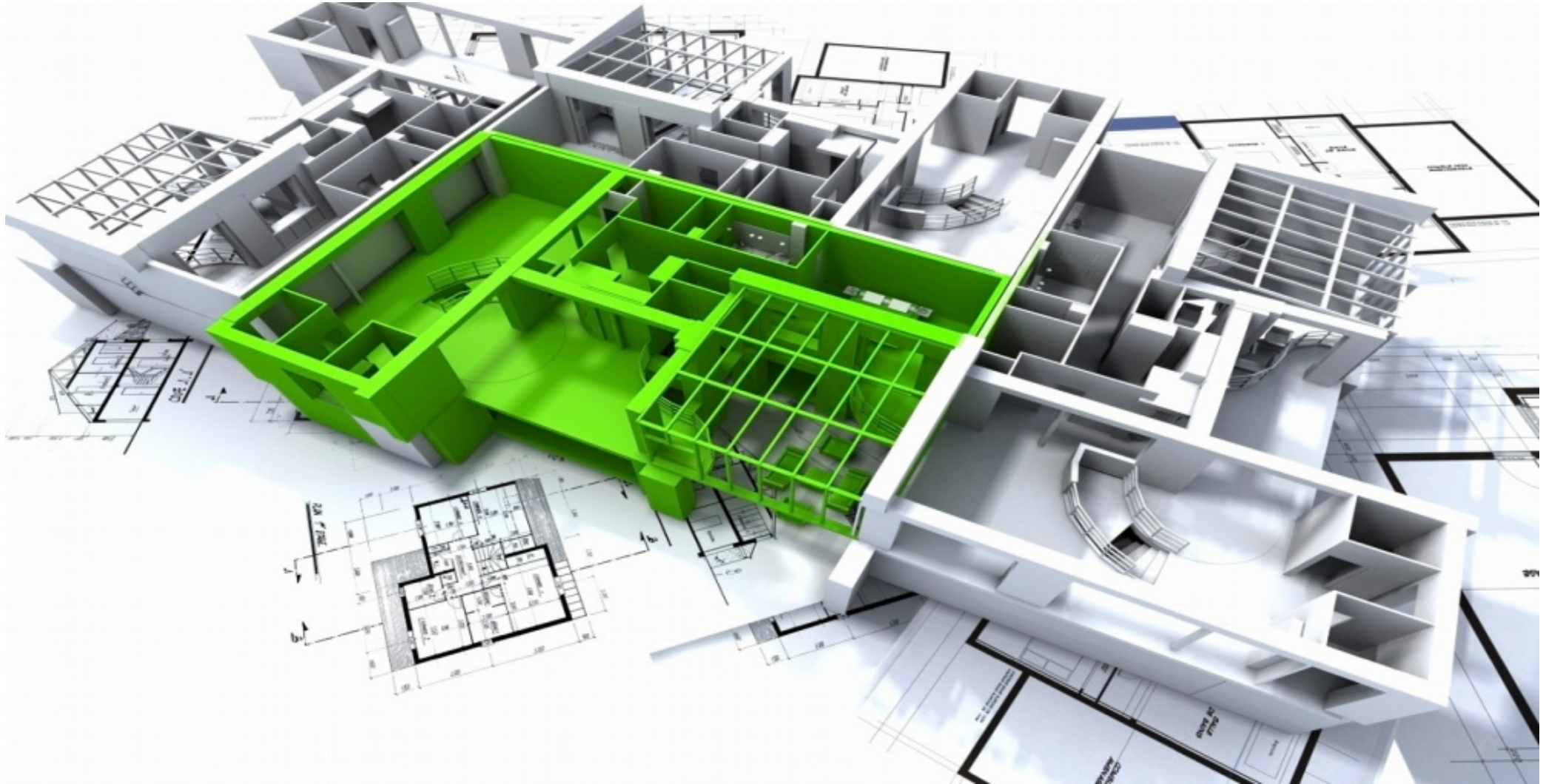
Bahubali 2, Arka Mediaworks and Tau Films, 2015

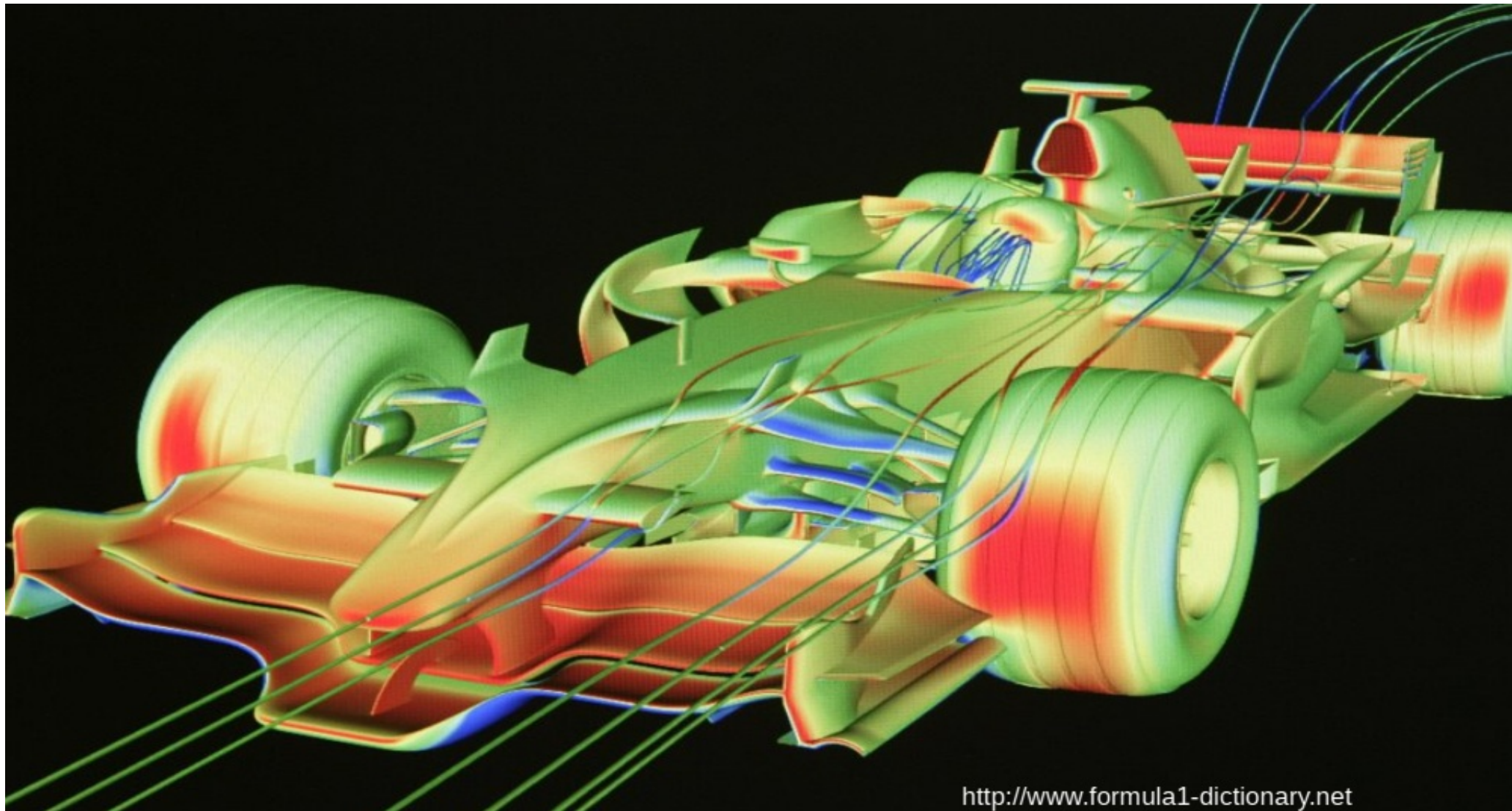


The Jungle Book, Disney, 2016



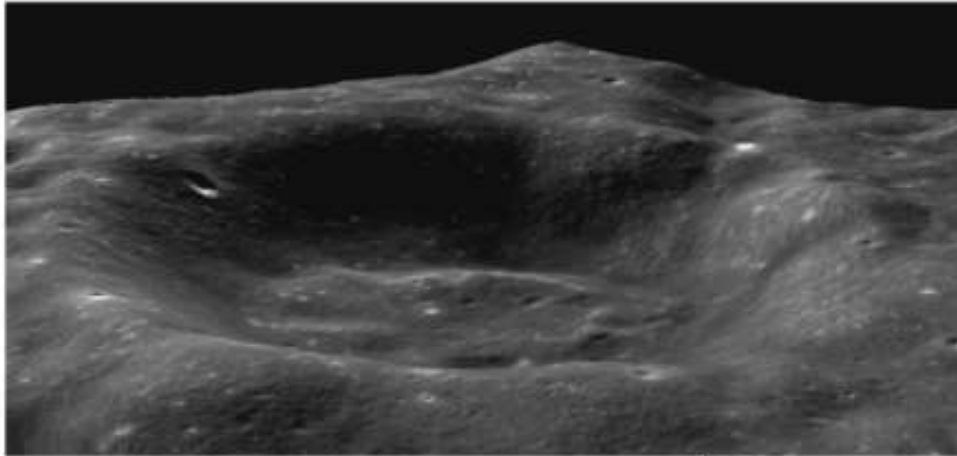
Age of Empires 3, Ensemble Studios, 2005





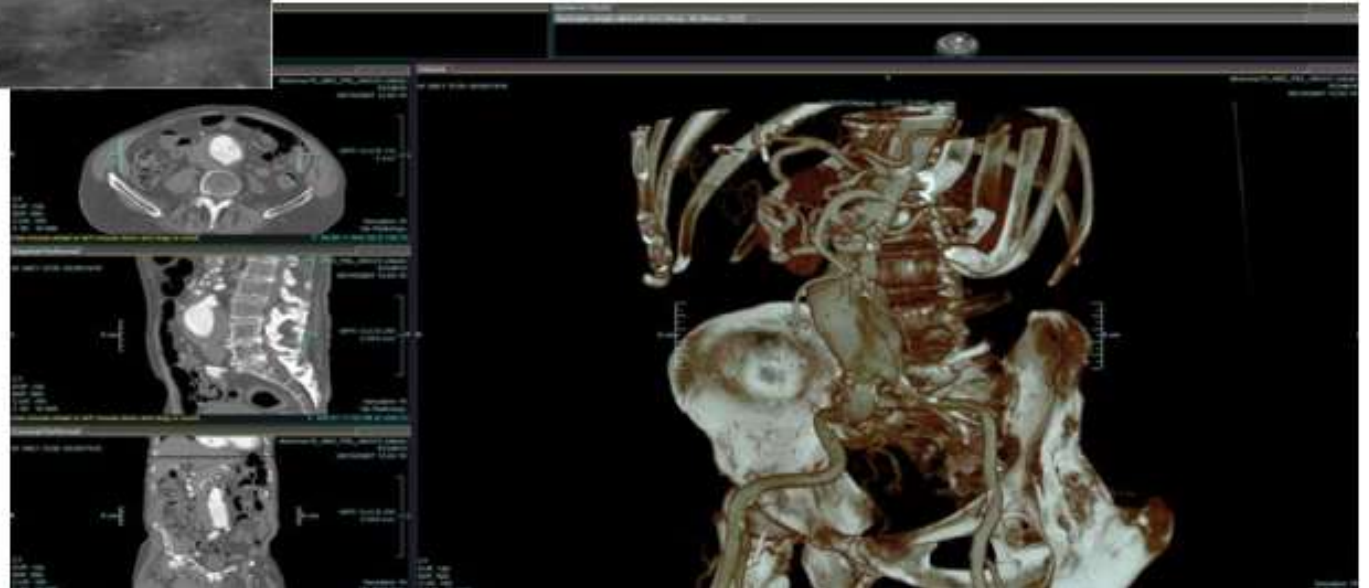


Scientific Visualization

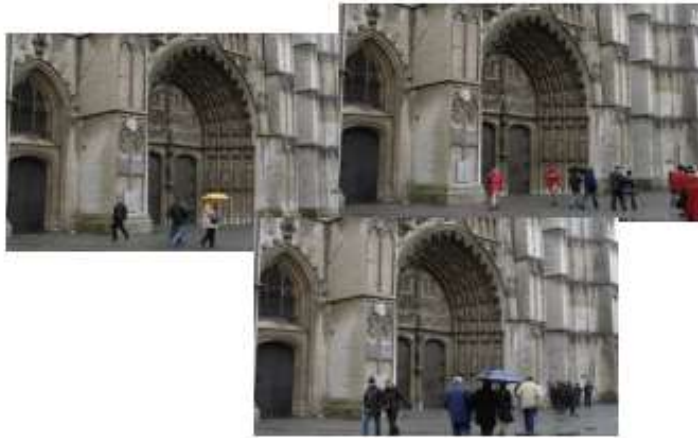


Chandrayaan, ISRO

Fiatlux Imaging

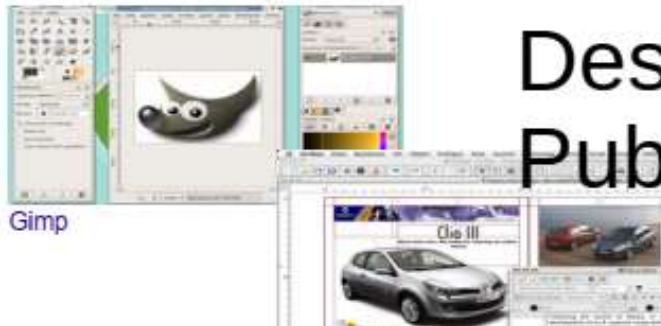


Digital and Print Media



Computational Photography

A Agarwala, M Dontcheva, M Agrawala, S Drucker, A Colburn, B Curless, D Salesin, M Cohen:
Interactive Digital Photomontage. ACM Transactions on Graphics (Proceedings of SIGGRAPH 2004)



Gimp

Desktop Publishing

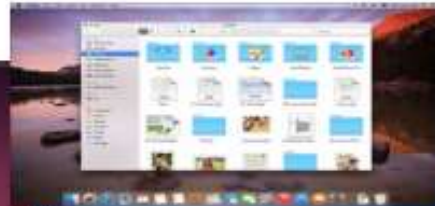


Networked Media

Graphical User Interfaces



Unity/Ubuntu



Aqua/Mac OS X 10.10



Metro/Windows 10

The Future?



© Twentieth Century Fox

Fraunhofer FIT



Now

Microsoft Surface Hub



Wacom Cintiq

Ivan Sutherland's Sketchpad



1962

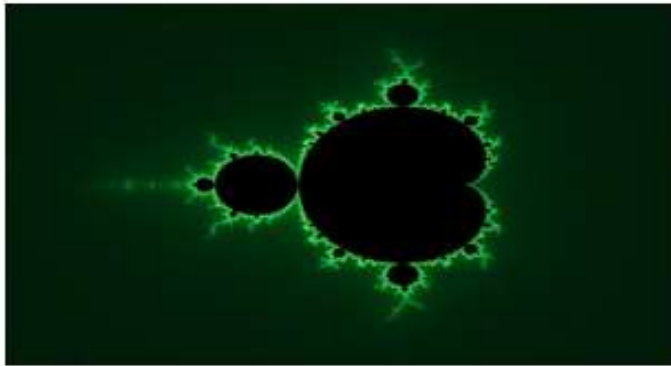
And the list goes on...

- Virtual Worlds – Google Earth, Second Life, Minecraft
- Algorithm Animation
- Typesetting and font design – LaTeX, Freetype
- Digital Video and HDTV
- Drug Design
- Image and Video Search
- ...

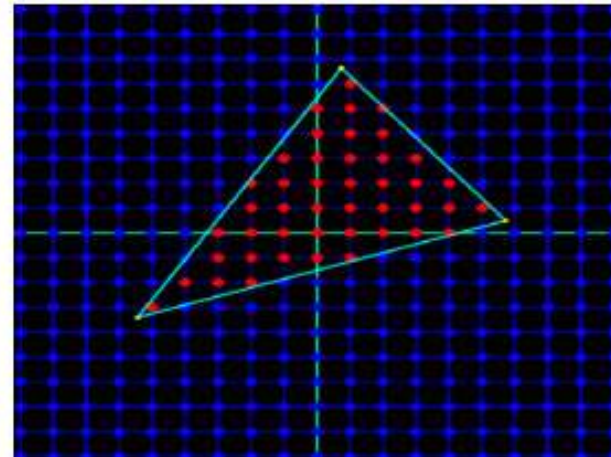
What is Computer Graphics?

- CG is the *art* and *science* of using the computer to make images.
- Study of methods (artistic, mathematical, algorithmic, software) and systems (mechanical, electronic, hardware) to create, control and manipulate pictorial data on the computer.

Drawing in 2D and 3D

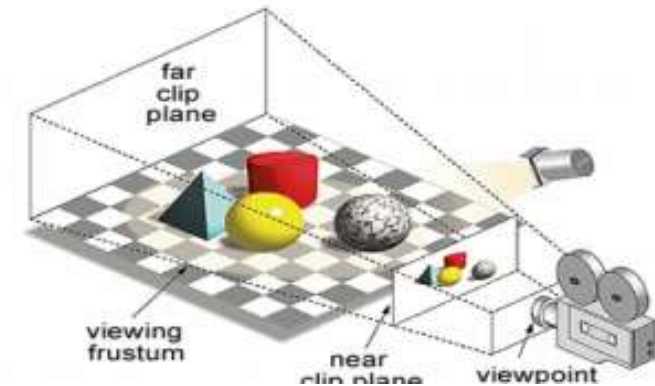
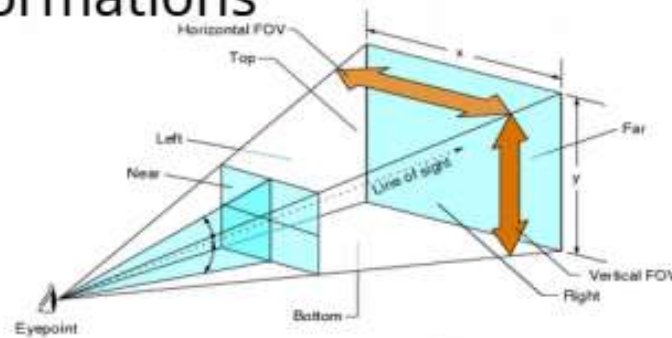
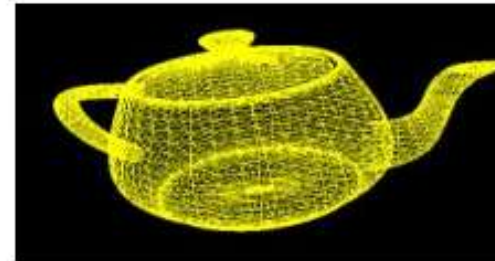


Colouring Pixels



Modelling

- Lines, Polygons
- Curves, Surfaces
- Modelling transformations
- Hidden Surfaces
- Viewing transformations



Rendering

- Shading
- Lighting models
- Texture mapping
- Ray Tracing



Animation

- Transformations
- Interpolation



Luxo Jr., PIXAR, 1986



Neha and Monica, CS675, 2010



Atul Rokade, CS675, 2012



Darshan Kapashi and Shivam Agarwal, CS475, 2011



Let's begin !

Application of Computer Graphics

Computer-Aided Design for engineering and architectural systems etc.

Objects may be displayed in a wireframe outline form. Multi-window environment is also favored for producing various zooming scales and views.

Animations are useful for testing performance.

Presentation Graphics

To produce illustrations which summarize various kinds of data. Except 2D, 3D graphics are good tools for reporting more complex data.

Computer Art

Painting packages are available. With cordless, pressure-sensitive stylus, artists can produce electronic paintings which simulate different brush strokes, brush widths, and colors. Photorealistic techniques, morphing and animations are very useful in commercial art. For films, 24 frames per second are required. For video monitor, 30 frames per second are required.

Entertainment

Motion pictures, Music videos, and TV shows, Computer games

Education and Training

Training with computer-generated models of specialized systems such as the training of ship captains and aircraft pilots.

Visualization

For analyzing scientific, engineering, medical and business data or behavior. Converting data to visual form can help to understand mass volume of data very efficiently.

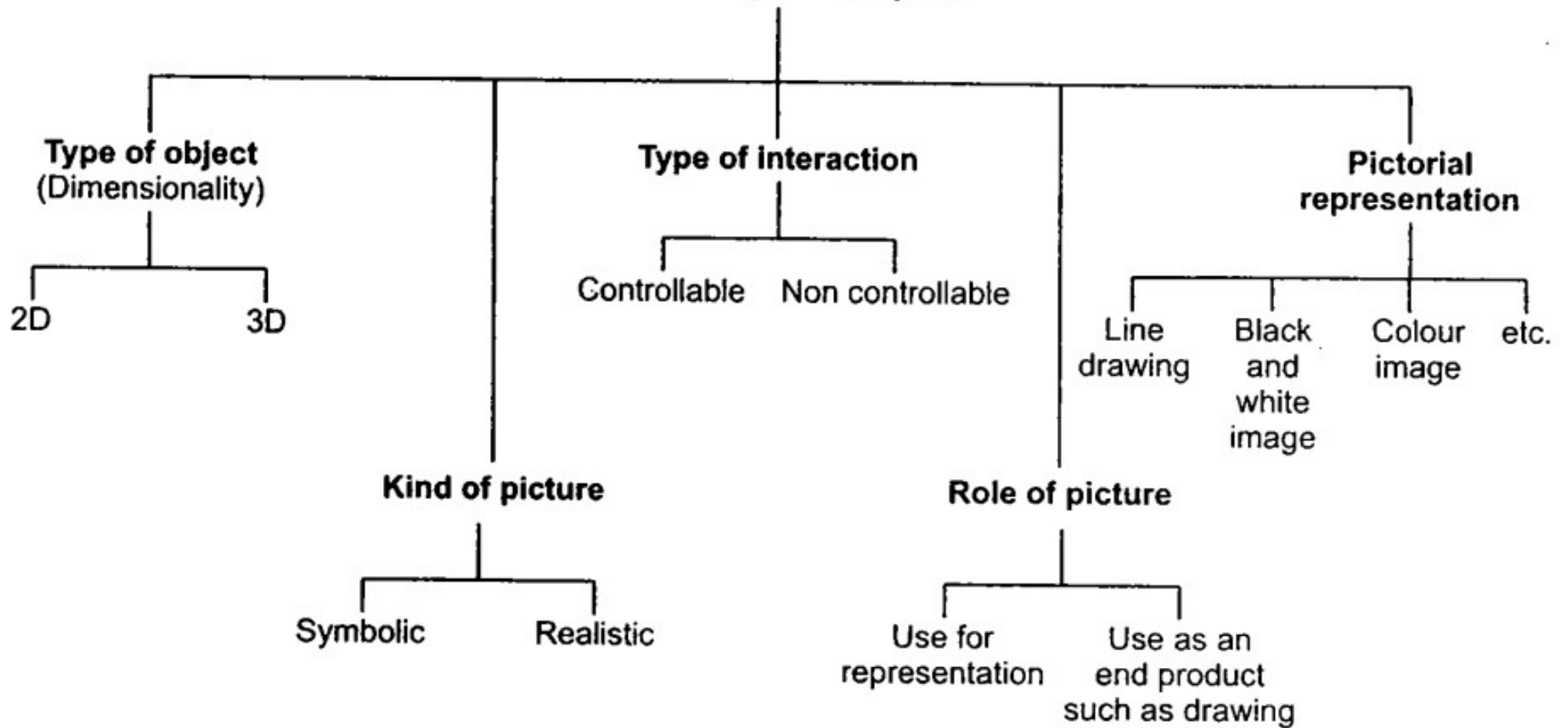
Image Processing

Image processing is to apply techniques to modify or interpret existing pictures. It is widely used in medical applications.

Graphical User Interface

Multiple window, icons, menus allow a computer setup to be utilized more efficiently

Uses of Computer Graphics



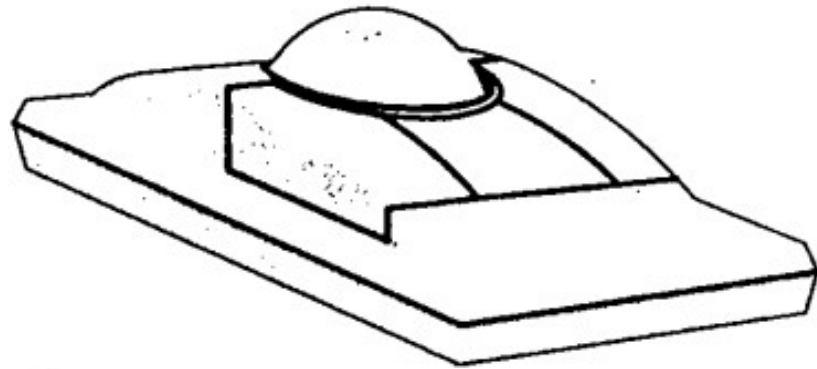
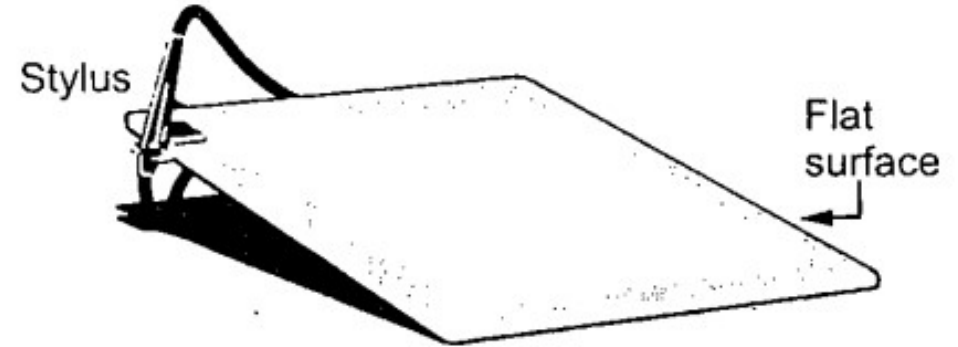


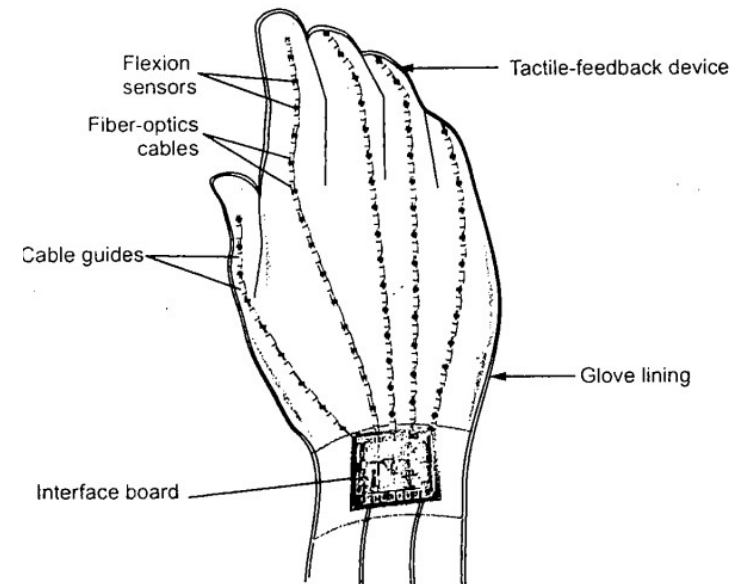
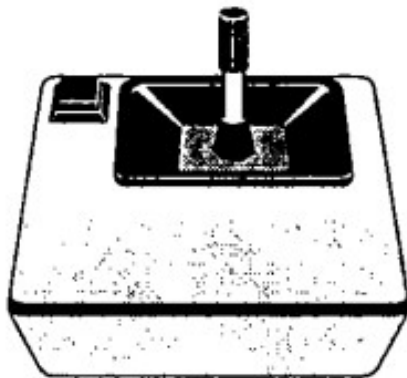
Image Scanners

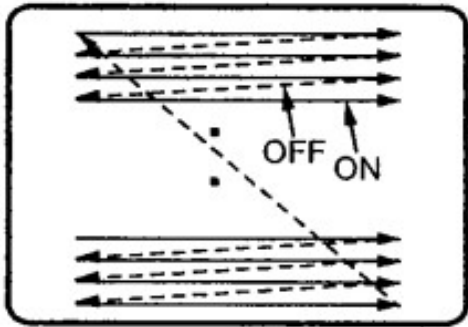
Touch Panels

Light Pens



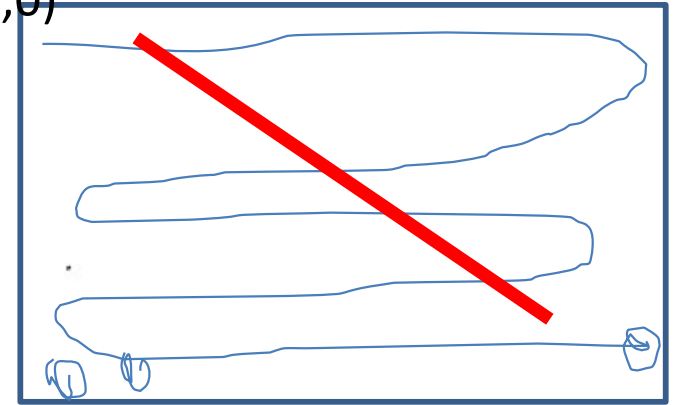
Input and output devices





1LED=1PIXEL

(0,0)



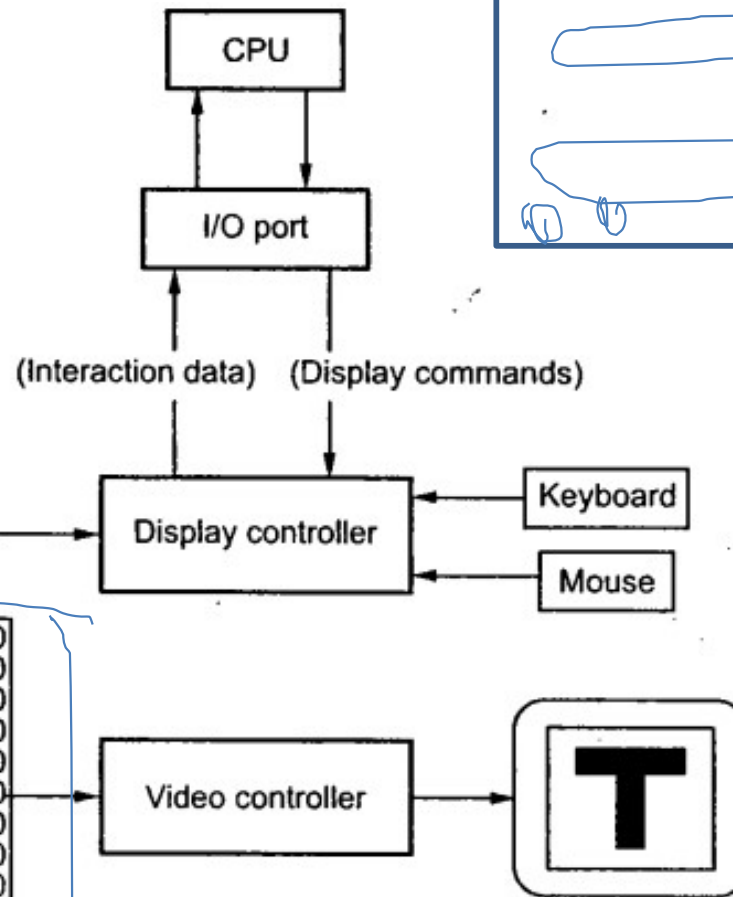
leds

PIXEL MAP

REFRESH RATE

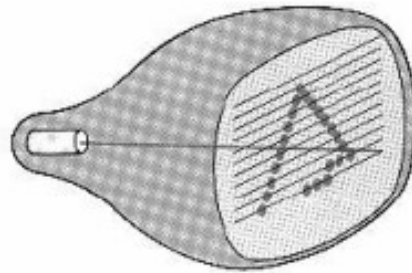
MEMORY IMAGE

Refresh buffer



Architecture of a raster display

Raster-Scan



The electron beam is swept across the screen one row at a time from top to bottom. As it moves across each row, the beam intensity is turned on and off to create a pattern of illuminated spots. This scanning process is called refreshing. Each complete scanning of a screen is normally called a **frame**.

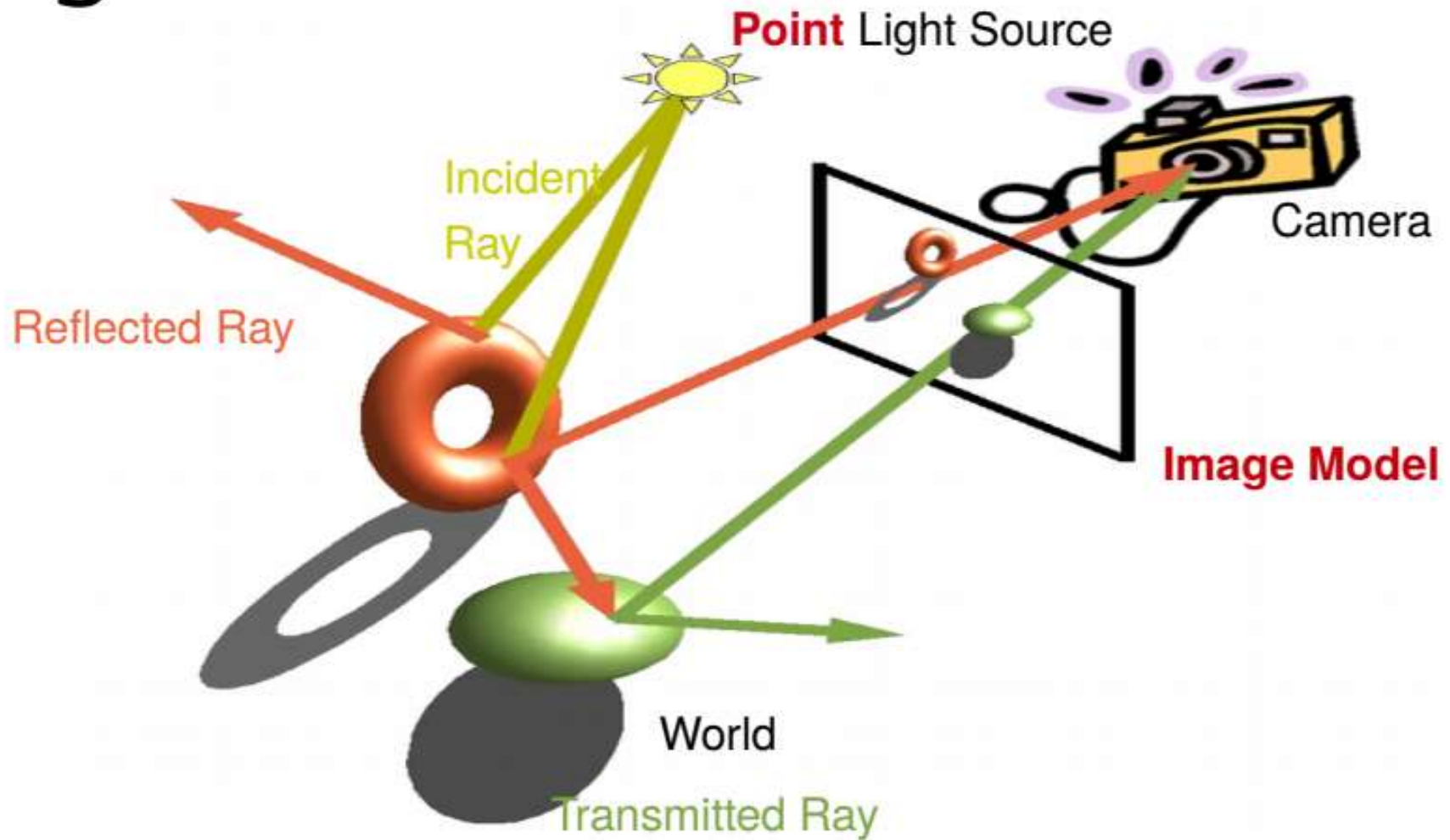
The refreshing rate, called the **frame rate**, is normally 60 to 80 frames per second, or described as 60 Hz to 80 Hz.

Picture definition is stored in a memory area called the **frame buffer**. This frame buffer stores the intensity values for all the screen points. Each screen point is called a **pixel** (picture element).

On black and white systems, the frame buffer storing the values of the pixels is called a **bitmap**. Each entry in the bitmap is a 1-bit data which determine the on (1) and off (0) of the intensity of the pixel.

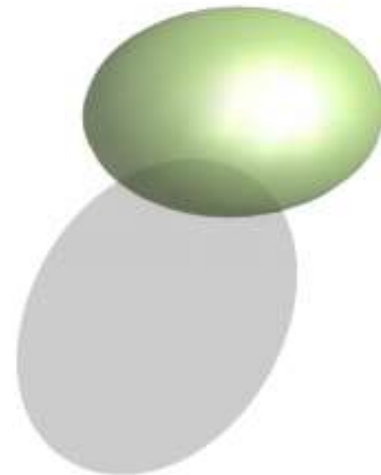
On color systems, the frame buffer storing the values of the pixels is called a **pixmap** (Though nowadays many graphics libraries name it as bitmap too). Each entry in the pixmap occupies a number of bits to represent the color of the pixel. For a true color display, the number of bits for each entry is 24 (8 bits per red/green/blue channel, each channel $2^8=256$ levels of intensity value, ie. 256 voltage settings for each of the red/green/blue electron guns).

Image Formation

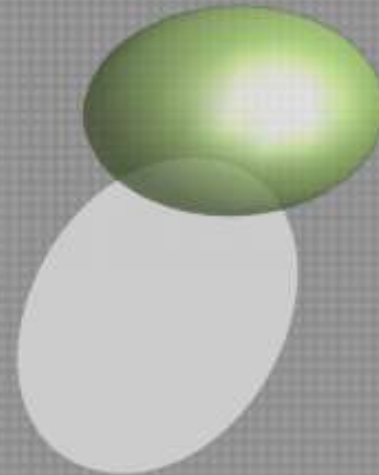
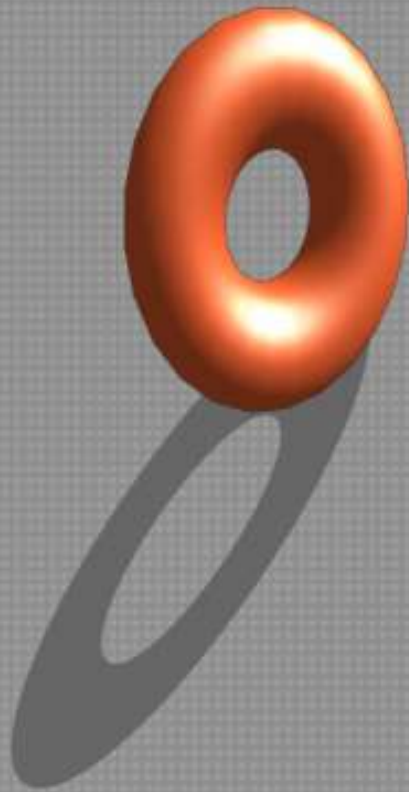


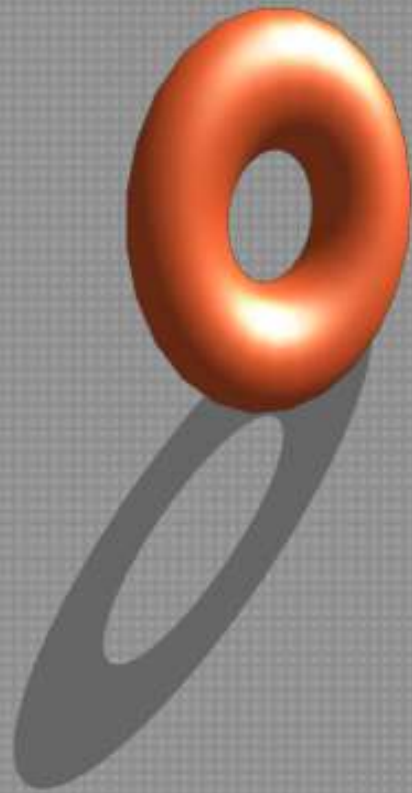


How is this image drawn
on the computer screen?

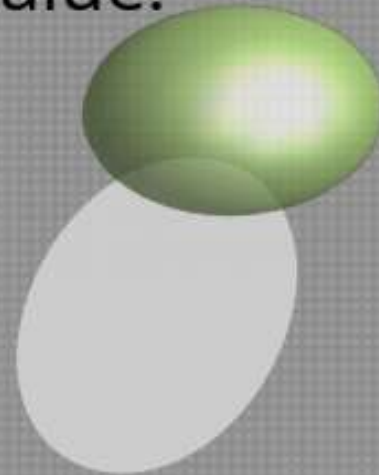


An image is an array of *raster* elements called *pixels*.

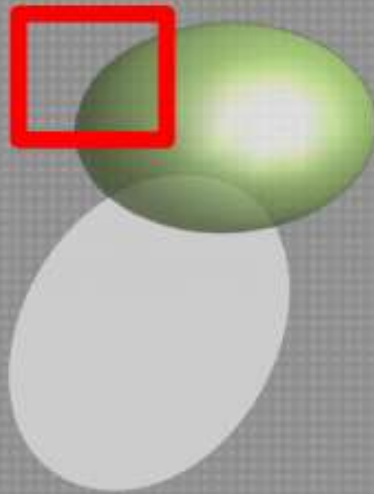




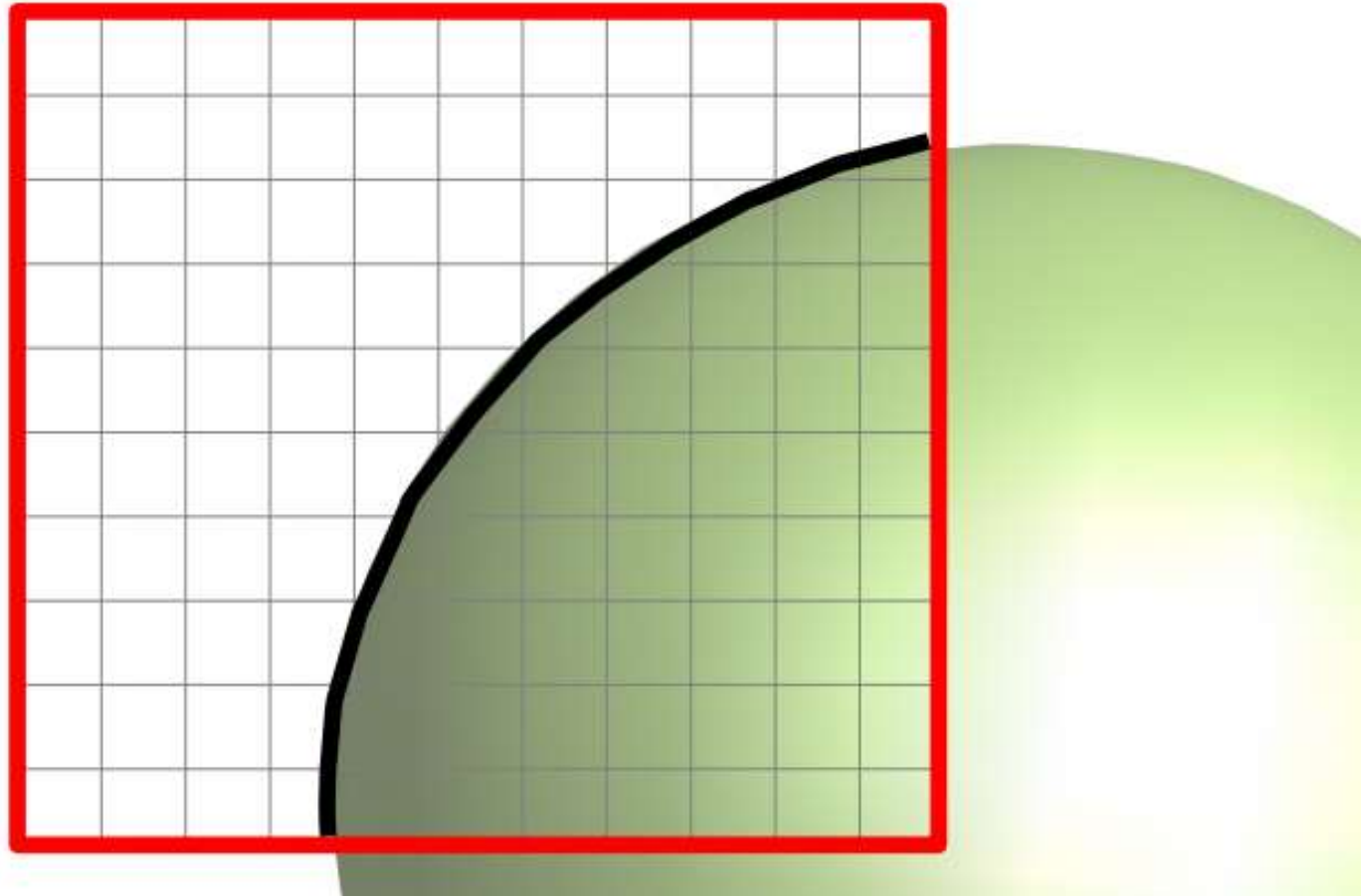
An image is an array of *raster* elements called *pixels*. Every pixel has at least a colour value.



Let's take a closer look.



How is the sphere drawn using the pixels?



To draw a geometrical figure...

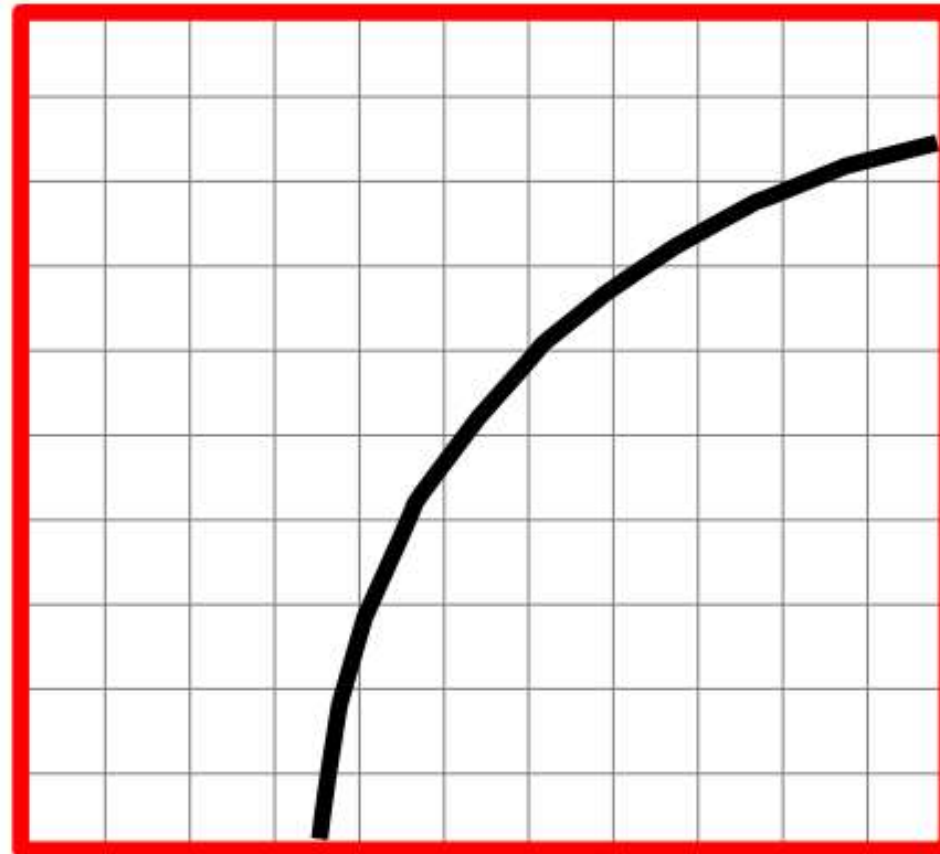
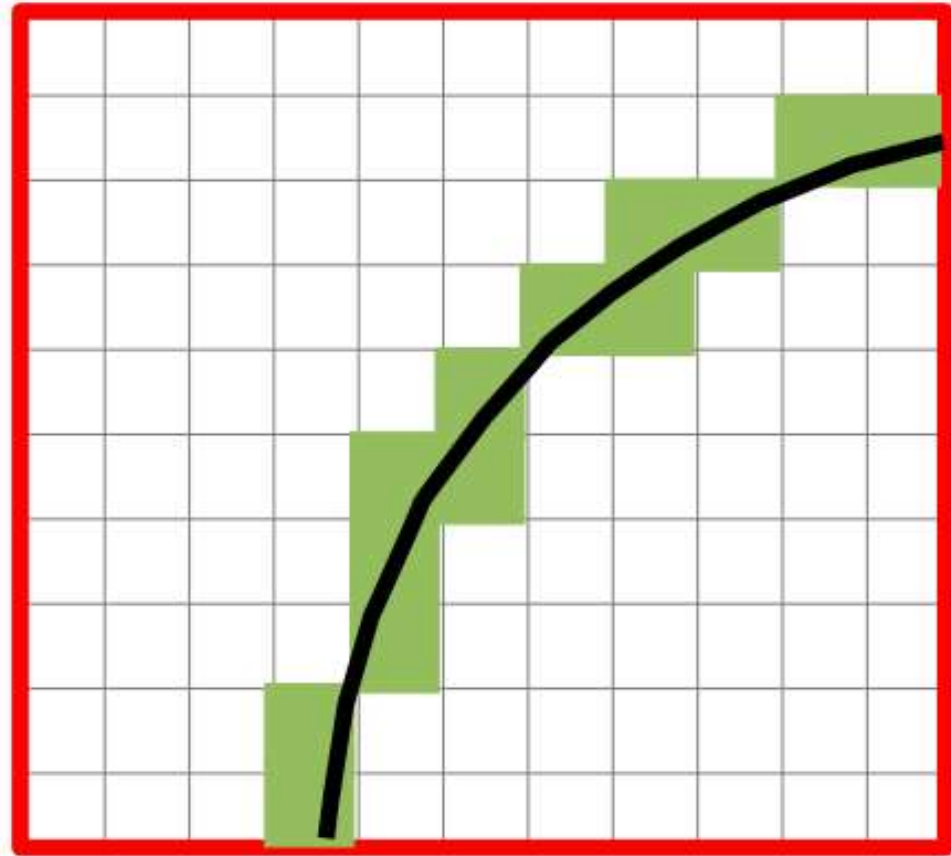


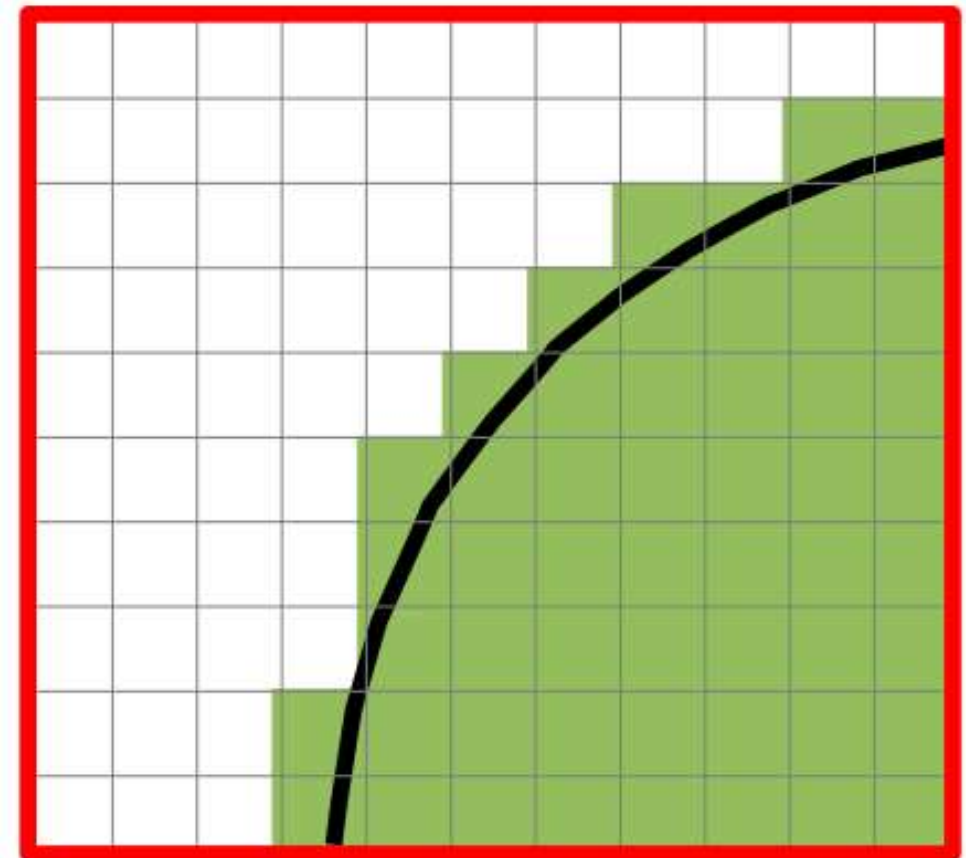
Image Model



...we assign the *correct* pixels with the correct colour. This process is called *rasterization*.

Continue the pixel colouring to get regions filled with colour.

The *framebuffer* is a memory buffer storing the colour value for each pixel displayed.



So the image model mimics the memory model from hardware.

Why this image model?



Display

Framebuffer



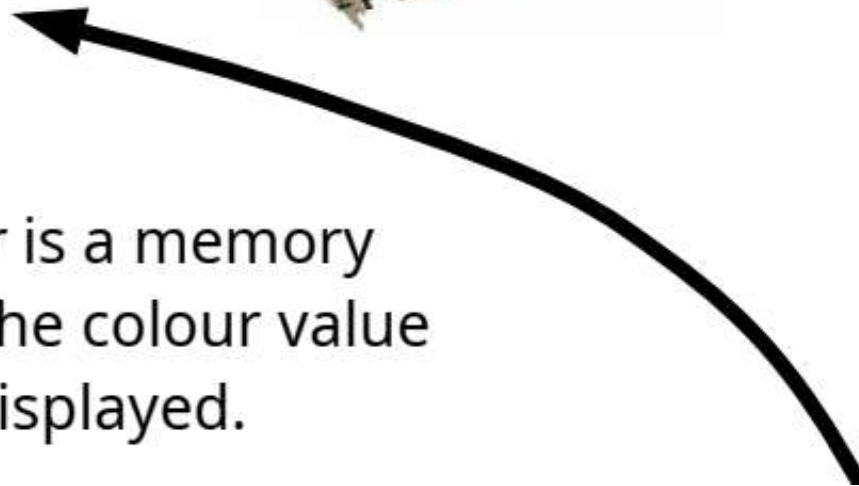
Processor



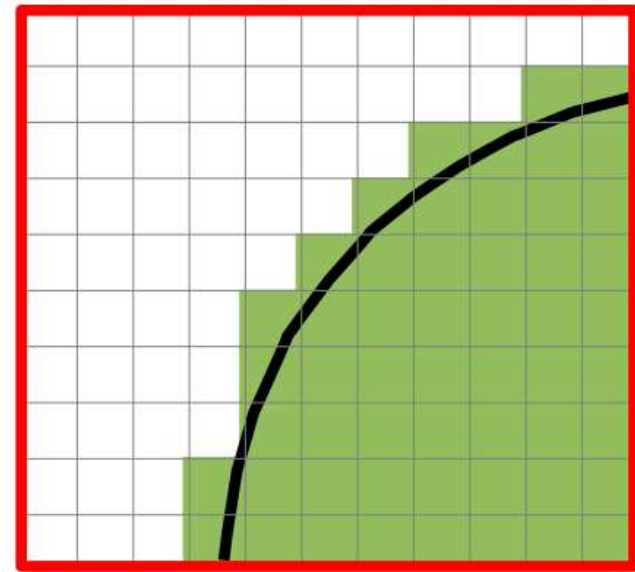
Input



The *framebuffer* is a memory buffer storing the colour value for each pixel displayed.



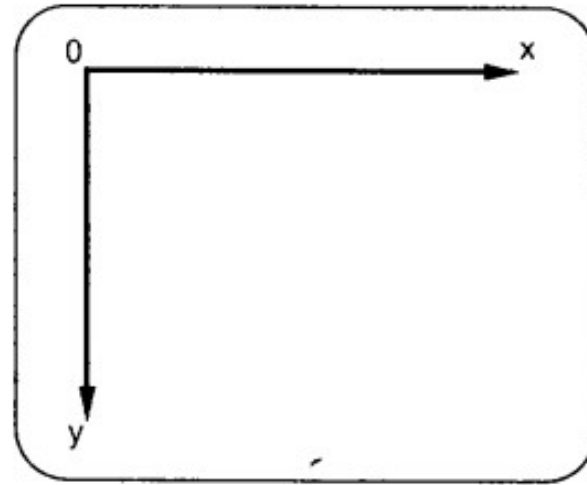
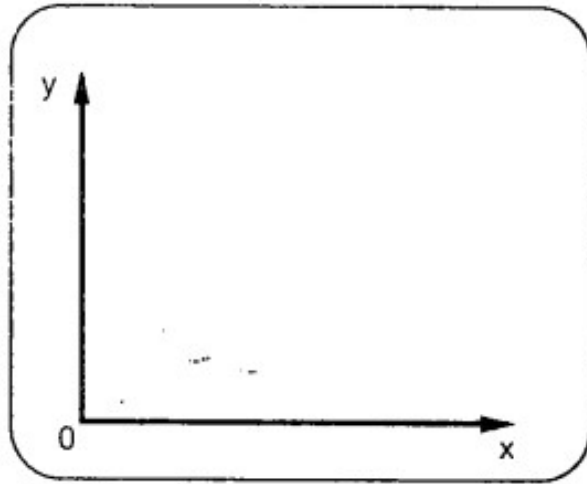
Anti-Aliasing



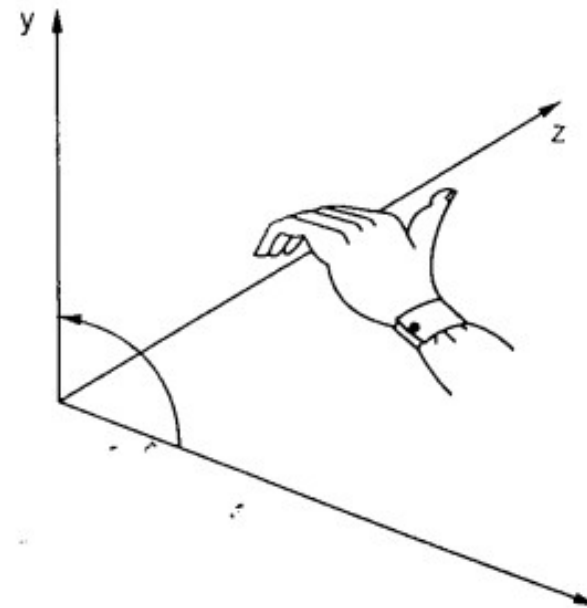
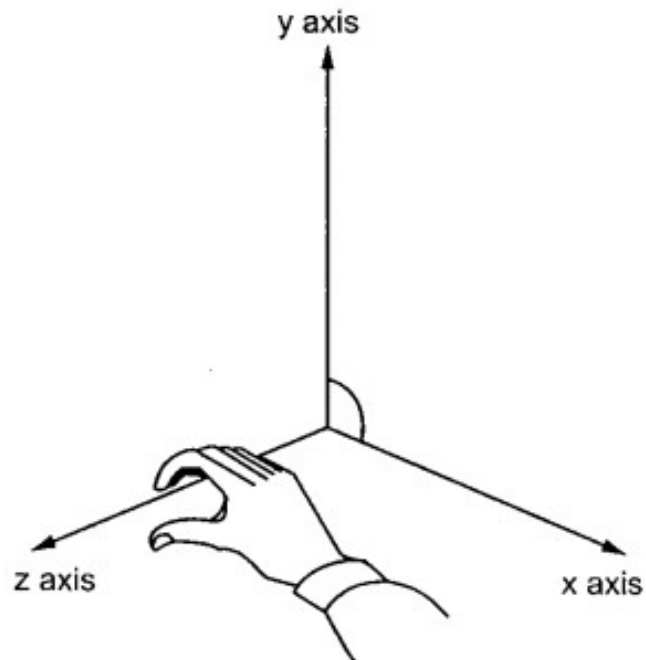
On dealing with integer pixel positions, jagged or stair step appearances happen very usually. This distortion of information due to under sampling is called aliasing. A number of ant aliasing methods have been developed to compensate this problem.

One way is to display objects at higher resolution. However there is a limit to how big we can make the frame buffer and still maintaining acceptable refresh rate.

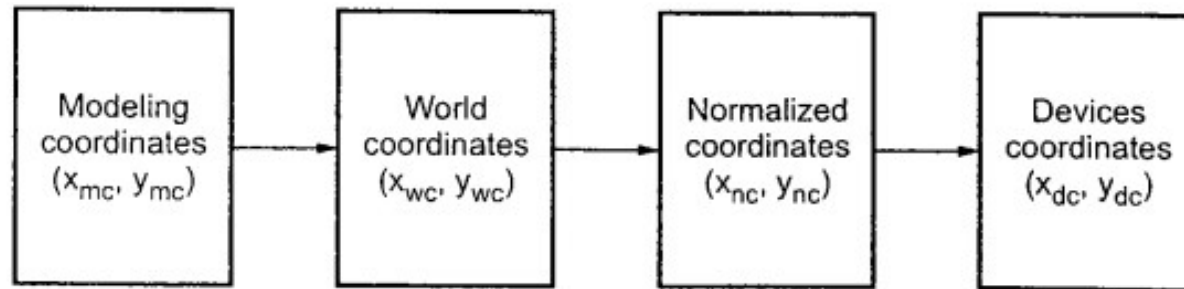
Coordinate Systems



Three Dimensional Cartesian Reference System



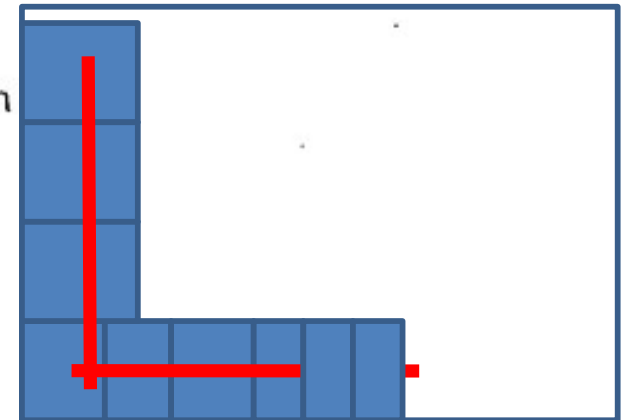
Coordinate Representations



A video monitor has a display area measuring 12 inch by 9.6 inch. If the resolution is 1280 by 1024 and the aspect ratio is 1. What is the diameter of each screen point ?

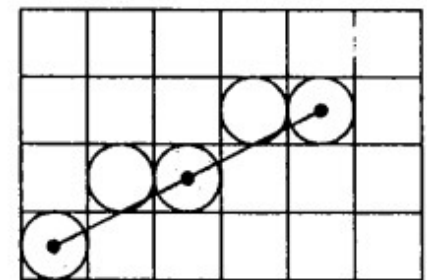
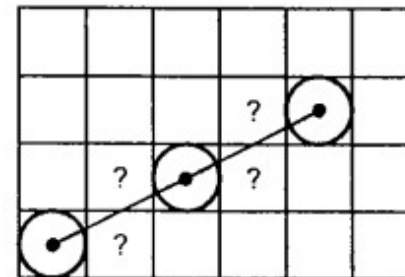
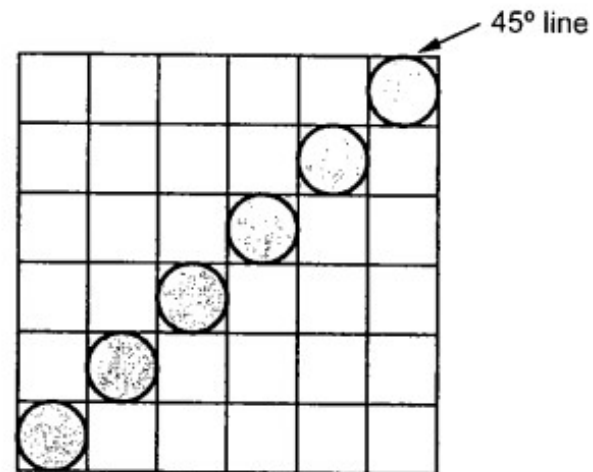
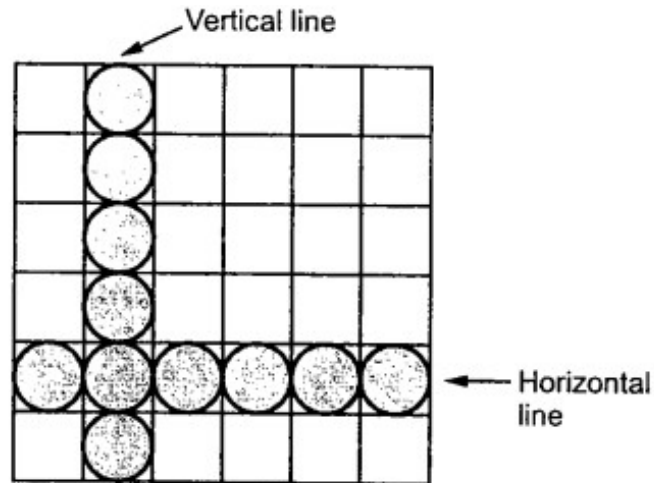
Sol.: An aspect ratio of 1 means that a vertical line plotted and horizontal line plotted with equal number of points have the same length. Therefore the diameter of each screen point can be given as

$$\begin{aligned} d &= \frac{\text{horizontal display length}}{\text{horizontal resolution}} \\ &= \frac{\text{vertical display length}}{\text{vertical resolution}} \\ &= \frac{12}{1280} = \frac{9.6}{1024} = 9.375 \times 10^{-3} \text{ inch} \end{aligned}$$

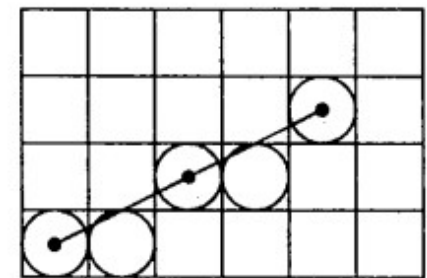


Drawing a Line

- The line should appear as a straight line and it should start and end accurately.
- The line should be displayed with constant brightness along its length independent of its length and orientation.
- The line should be drawn rapidly.



OR



DDA Algorithm

Digital Differential Analyzer

$$Y = MX + B$$

$$m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$

$$\Delta y = \frac{y_2 - y_1}{x_2 - x_1} \Delta x$$

$$\Delta x = \frac{x_2 - x_1}{y_2 - y_1} \Delta y$$

$$\begin{aligned} x_{i+1} &= x_i + \Delta x \\ &= x_i + \frac{x_2 - x_1}{y_2 - y_1} \Delta y \end{aligned}$$

$$\begin{aligned} y_{i+1} &= y_i + \Delta y \\ &= y_i + \frac{y_2 - y_1}{x_2 - x_1} \Delta x \end{aligned}$$

if $|\Delta x| \geq |\Delta y|$ then $\Delta x = 1$

else $\Delta y = 1$

if $\Delta x = 1$ then

$$y_{i+1} = y_i + \frac{y_2 - y_1}{x_2 - x_1} \text{ and}$$

$$x_{i+1} = x_i + 1$$

If $\Delta y = 1$ then

$$\begin{aligned} y_{i+1} &= y_i + 1 \text{ and} \\ x_{i+1} &= x_i + \frac{x_2 - x_1}{y_2 - y_1} \end{aligned}$$

DDA Line Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.
[if equal then plot that point and exit]
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. if $(\Delta x \geq \Delta y)$ then
 length = Δx
else
 length = Δy
end if
4. $\Delta x = (x_2 - x_1) / \text{length}$
 $\Delta y = (y_2 - y_1) / \text{length}$
 [This makes either Δx or Δy equal to 1 because length is either $|x_2 - x_1|$ or $|y_2 - y_1|$. Therefore, the incremental value for either x or y is one.]
5. $x = x_1 + 0.5 * \text{Sign}(\Delta x)$
 $y = y_1 + 0.5 * \text{Sign}(\Delta y)$
 [Here, Sign function makes the algorithm work in all quadrant. It returns - 1, 0, 1 depending on whether its argument is < 0 , $= 0$, > 0 respectively. The factor 0.5 makes it possible to round the values in the integer function rather than truncating them.]
6. $i = 1$ [Begins the loop, in this loop points are plotted]
 While ($i \leq \text{length}$)
 {
 Plot (Integer (x), Integer (y))
 $x = x + \Delta x$
 $y = y + \Delta y$
 $i = i + 1$
 }
7. Stop

```

#include<stdio.h>
#include<graphics.h>
#include<math.h>
main()
{
float x,y,x1,y1,x2,y2,dx,dy,length;
int i,gd,gm;
clrscr();

/* Read two end points of line
----- */
printf("Enter the value of x1 :\t");
scanf("%f",&x1);

printf("Enter the value of y1 :\t");
scanf("%f",&y1);
printf("Enter the value of x2 :\t");
scanf("%f",&x2);
printf("Enter the value of y2 :\t");
scanf("%f",&y2);

```

```

detectgraph(&gd,&gm);
initgraph(&gd,&gm,"");

dx=abs(x2-x1);
dy=abs(y2-y1);

if (dx >= dy)
{
length = dx;
}
else
{
length = dy;
}

dx = (x2-x1)/length;
dy = (y2-y1)/length;

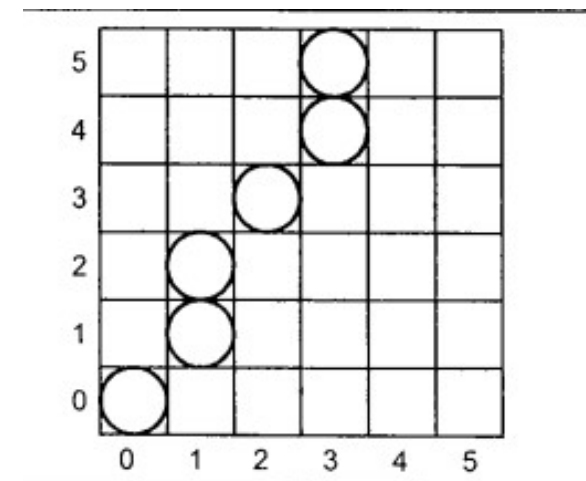
x = x1 + 0.5; /* Factor 0.5 is added to round the values */
y = y1 + 0.5; /* Factor 0.5 is added to round the values */

i = 1; /* Initialise loop counter */
while(i <= length)
{
putpixel(x,y,15);
x = x + dx;
y = y + dy;
i = i + 1;
delay(100); /* Delay is purposely inserted to see
observe the line drawing process */

```

Consider the line from $(0, 0)$ to $(4, 6)$. Use the simple DDA algorithm to rasterize this line

i	Plot	x	y
1	$(0, 0)$	0.5	0.5
2	$(1, 1)$	1.167	1.5
3	$(1, 2)$	1.833	2.5
4	$(2, 3)$	2.5	3.5
5	$(3, 4)$	3.167	4.5
6	$(3, 5)$	3.833	5.5
		4.5	6.5





$$DX = x_2 - x_1 = 4$$

$$Dy = y_2 - y_1 = 6 - 0 = 6$$

Length = 6-----DY>DX

$$dx = 4/6 = x_2 - x_1 / \text{length}$$

$$dy = 6/6 = 1$$

$$X = x_0 + 0.5 (+) = 0.5$$

$$Y = y_0 + 0.5 (+) = 0.5$$

$$X_2 = x_1 + dx = 0.5 + 4/6 = 1.167$$

$$Y_2 = y_1 + dy = 0.5 + 1 = 1.5$$

$$X_3 = x_2 + dx = 1.167 + 4/6 = 1.84$$

$$Y_3 = y_2 + dy = 1.5 + 1 = 2.5$$

$$X_4 = 1.84 + 4/6 = 2.5$$

$$Y_4 = 2.5 + 1 = 3.5$$

$$X_5 = 2.5 + 4/6 = 3.16$$

$$Y_5 = 4.5$$

$$X_6 = 3.16 + 4/6 = 3.77$$

$$Y_6 = 5.5$$

$$X_7 = 3.77 + 4/6 = 4.45$$

$$Y_7 = 6.5$$

Ploted

(0.5,0.5)

(1.167,1.5)

(1.84,2.5)

(2.5,3.5)

3.16,4.5)

3.77,5.5)

(4.45,6.5)

Putpixel(0.5,0.5) -----(0,0)

(0,0)

(1,1)

(1,2)

2,3

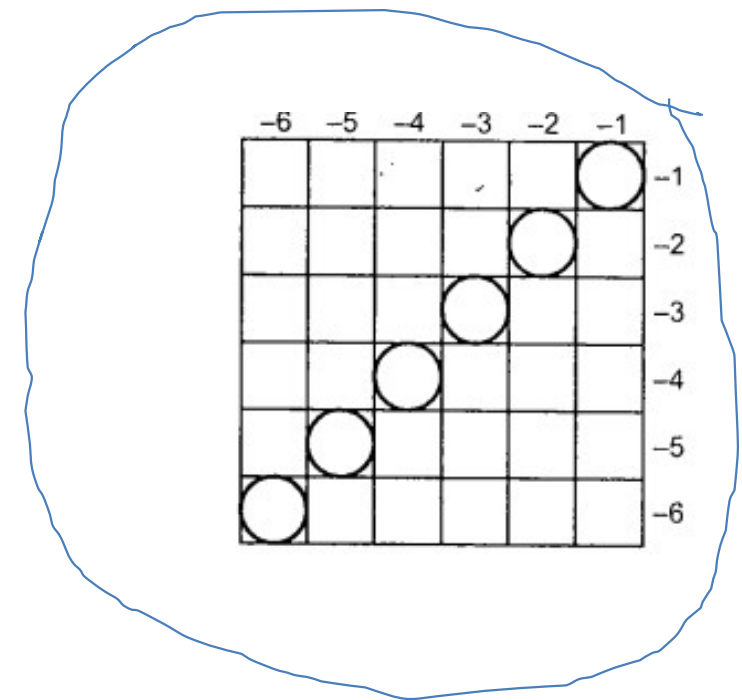
3,4

3,5

4,6

Consider the line from $(0, 0)$ to $(-6, -6)$. Use the simple DDA algorithm to rasterize this line.

i	Plot	x	y
1	$(-1, -1)$	-0.5	-0.5
2	$(-2, -2)$	-1.5	-1.5
3	$(-3, -3)$	-2.5	-2.5
		-3.5	-3.5
5	$(-5, -5)$	-4.5	-4.5
		-5.5	-5.5
6	$(-6, -6)$	-6.5	-6.5



$DX = 6 \quad !-6-0! === -ve$

$DY = 6 \quad ===== -ve$

Length = 6

$dx = DX/length = -6/6 = -1$

$dy = DY/length = -6/6 = -1$

$X = x0 + 0.5 (-) = 0 - 0.5 = -0.5$

$Y = y0 + 0.5 (-) = 0 - 0.5 = -0.5$

$x2 = x1 + dx = -0.5 - 1 = -1.5$

$Y2 = y1 + dy = -0.5 - 1 = -1.5$

$X3 = -1.5 - 1 = -2.5$

$Y3 = -1.5 - 1 = -2.5$

$X4 = -2.5 - 1 = -3.5$

$Y4 = -2.5 - 1 = -3.5$

$X5 = -3.5 - 1 = -4.5$

$Y5 = -3.5 - 1 = -4.5$

$X6 = -4.5 - 1 = -5.5$

$Y6 = -4.5 - 1 = -5.5$

$X7 = -5.5 - 1 = -6.5$

$Y7 = -5.5 - 1 = -6.5$

$(-0.5, -0.5)$

$(-1.5, -1.5)$

$(-2.5, -2.5)$

$(-3.5, -3.5)$

$(-4.5, -4.5)$

$(-5.5, -5.5)$

$(-6.5, -6.5)$

Pixel to be made ON

$(0,0)$

$-1,-1$

$-2,-2$

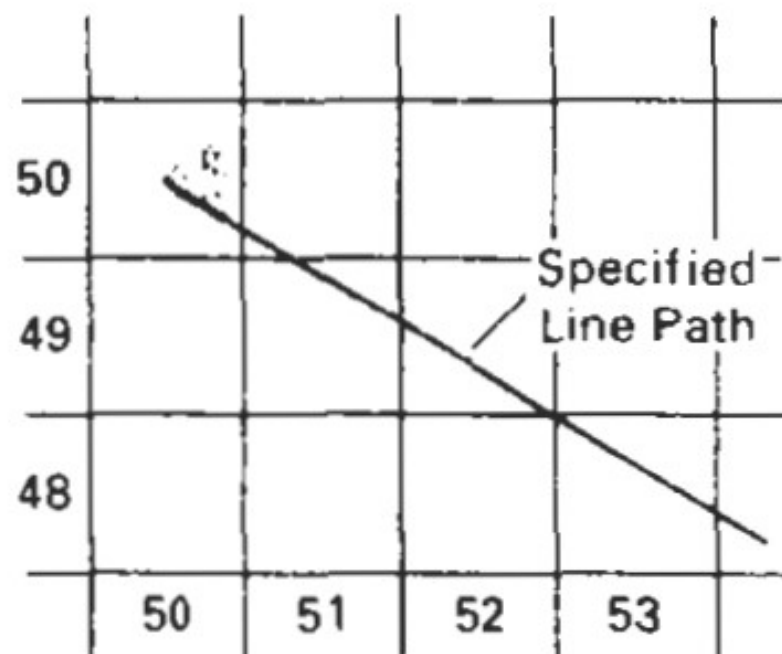
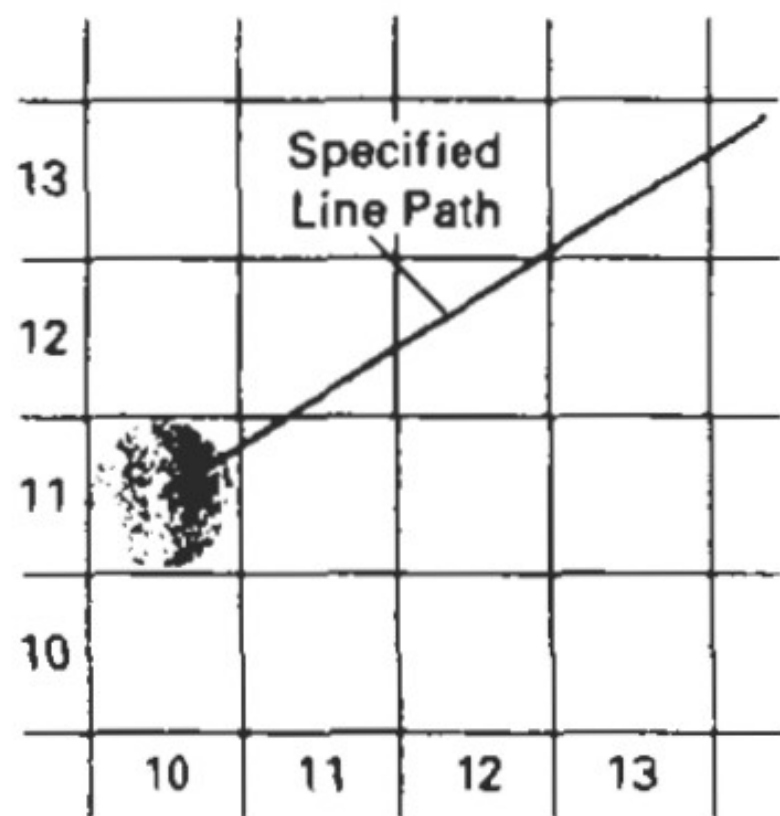
$-3,-3$

$-4,-4$

$-5,-5$

$-6,-6$

Bresenham's Line Algorithm

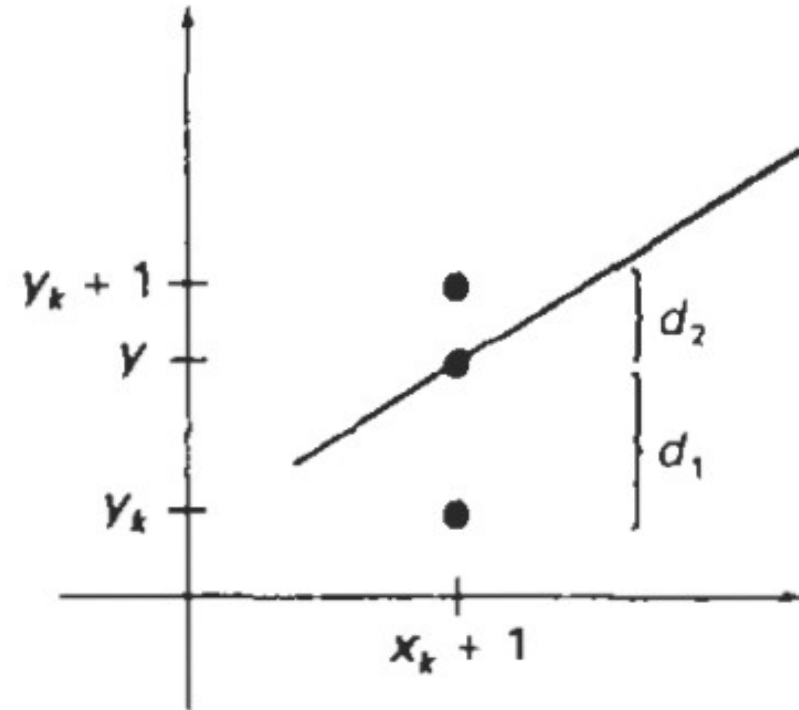
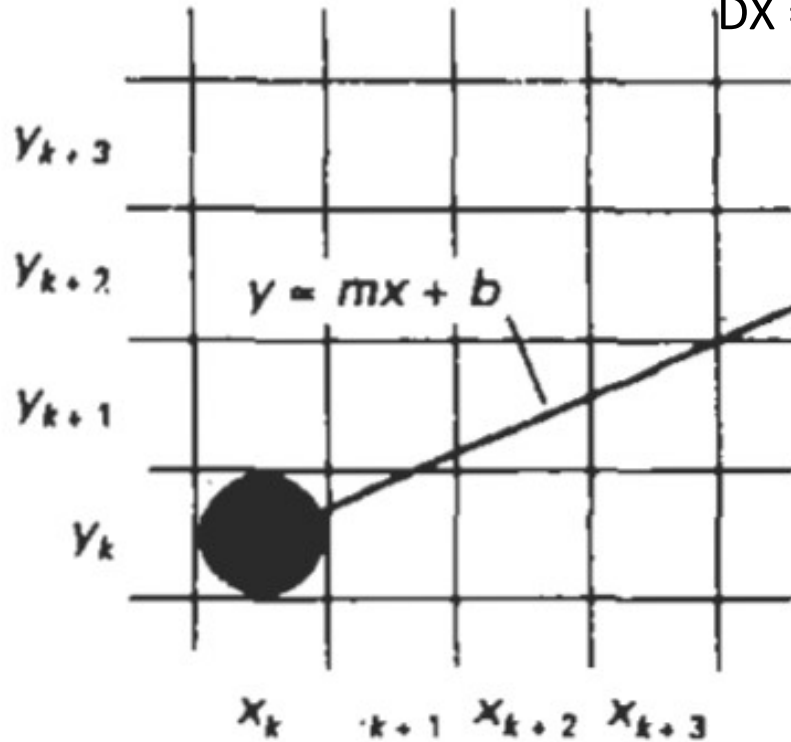


$$d1 = y - y_k = mx + b - y_k$$

$$d2 = y_{k+1} - y = y_{k+1} - mx - b$$

$P_k = DX (d1 - d2)$ -----we have defined this

$$DX = X_2 - X_1$$



$$P_k = DX (m(x_{k+1}) + b - y_k - \{y_{k+1} - m(x_{k+1}) - b\})$$

$$d1 - d2 = m(x_{k+1}) + b - y_k - \{y_{k+1} - m(x_{k+1}) - b\}$$

$$PK = DX (m(x_{k+1}) + b - y_k - \{y_{k+1} - m(x_{k+1}) - b\})$$

$$m = DY/DX$$

$$P_k = DX (d_1 - d_2)$$

$$P_k = 2DY X_k - 2DX Y_k + 2DY + 2b DX - DX$$

$$P_{k+1} = 2DY X_{k+1} - 2DX Y_{k+1} + 2DY + 2b DX - DX$$

$$P_{k+1} - P_k = 2DY - 2DX(Y_{k+1} - Y_k)$$

$$P_{k+1} = P_k + 2DY - 2DX(Y_{k+1} - Y_k) \quad \text{0 OR 1}$$

If $d_1 < d_2$; p_k is -ve
plot (x_{k+1}, y_k)

DEPENDING ON SIGN OF PK

IF $PK < 0$,, $d_1 < d_2$ then $(Y_{k+1} - Y_k) = 0$

Plot (x_{k+1}, y_k)

$$P_{k+1} = P_k + 2DY$$

If $d_1 > d_2$; p_k is +ve
Plot (x_{k+1}, y_{k+1})

IF $PK > 0$,, $d_1 < d_2$ then $(Y_{k+1} - Y_k) = 1$

Plot (x_{k+1}, y_{k+1})

$$P_{k+1} = P_k + 2DY - 2DX$$

Bresenham's Line-Drawing Algorithm for $|m| < 1$

1. Input the two line endpoints and store the left endpoint in (x_0, y_0) .
2. Load (x_0, y_0) into the frame buffer; that is, plot the first point.
3. Calculate constants Δx , Δy , $2\Delta y$, and $2\Delta y - 2\Delta x$, and obtain the starting value for the decision parameter as

$$p_0 = 2\Delta y - \Delta x$$

4. At each x_k along the line, starting at $k = 0$, perform the following test:
If $p_k < 0$, the next point to plot is $(x_k + 1, y_k)$ and

$$p_{k+1} = p_k + 2\Delta y$$

Otherwise, the next point to plot is $(x_k + 1, y_k + 1)$ and

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

5. Repeat step 4 Δx times.

Scan convert by Bresenham's algorithm

1) **20,10 to 30,18**

2) 5,5 to 13,9

$$P_0 = 2(8) - 10 = 6 > 0$$

$$P_1 = P_0 + 2 DY - 2 DX = 6 + 2(8) - 2(10) = 2 > 0$$

$$P_2 = P_1 + 2 DY - 2 DX = 2 + 16 - 20 = -2 < 0$$

$$P_3 = P_2 + 2 DY = -2 + 16 = 14 > 0$$

$$P_4 = P_3 + 2 DY - 2 DX = 14 + 16 - 20 = 10 > 0$$

$$P_5 = P_4 + 2 DY - 2 DX = 10 + 16 - 20 = 6 > 0$$

$$P_6 = P_5 + 2 DY - 2 DX = 6 + 16 - 20 = 2 > 0$$

$$P_7 = 2 + 16 - 20 = -2 < 0$$

$$P_8 = -2 + 16 = 14 > 0$$

$$P_9 = 14 + 16 - 20 = 10 > 0$$

(20,10)

(21, 11)

(22,12)

(23, 12)

(24,13)

(25, 14)

26,15

27 ,16)

28,16

29,17

30,18



k	p_k	(x_{k+1}, y_{k+1})	k	p_k	(x_{k+1}, y_{k+1})
0	6	(21, 11)	5	6	(26, 15)
1	2	(22, 12)	6	2	(27, 16)
2	-2	(23, 12)	7	-2	(28, 16)
3	14	(24, 13)	8	14	(29, 17)
4	10	(25, 14)	9	10	(30, 18)

i	Plot	x	y	e
		5	5	0
1	(5, 5)	6	6	-8
2	(6, 6)	7	6	0
3	(7, 6)	8	7	-8
4	(8, 7)	9	7	0
5	(9, 7)	10	8	-8
6	(10, 8)	11	8	0
7	(11, 8)	12	9	-8
8	(12, 9)	13	9	0
9	(13, 9)	14	10	-8